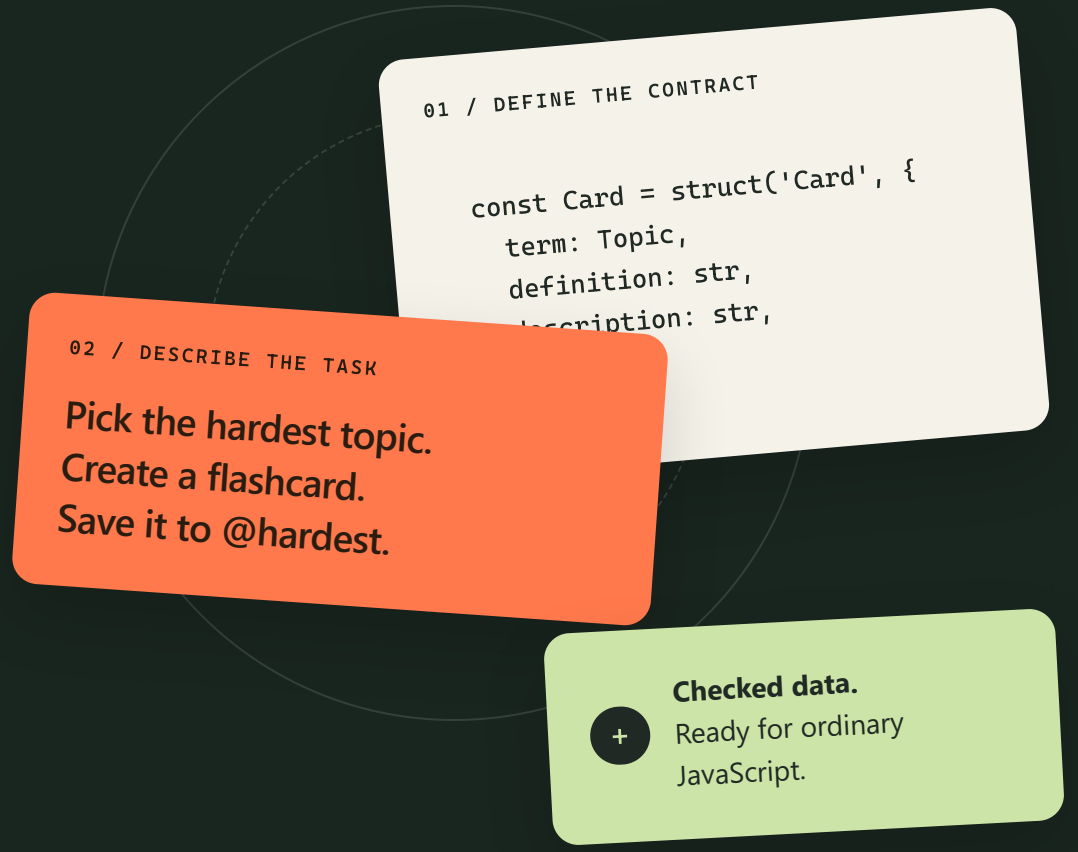
Write the code. Describe the *judgment.*

Build AI workflows in one readable file.
Native JavaScript where you need precision.
Natural language where you need a model.

Executable Markdown | .spl blocks | Node.js library



One document.
One shared scope.

Steps run top to bottom. A variable created in code or saved by an agent is available to the next step.

01

Set things up in code.

Read inputs, declare types, import modules, calculate values. Top-level `await` works.

02

Give the agent a task.

A blockquote acts through tools. `@name` references a shared binding; it is not string substitution.

03

Use the checked result.

Assert business rules, format Markdown, write a file or finish with `success(value)`.

.md ```` ```js ... ``` ```` + blockquotes; frontmatter required

.spl `{js}` `{agent}` Same workflow, explicit blocks

Headings and ordinary prose are commentary. Executable steps are not.

Start small.
Then add a model.

1 **Install on Node.js 24.15+.**

The CLI runs on Windows, macOS and Linux.

2 **Run a no-model example.**

Explore types and ordinary JavaScript before spending a token.

3 **Configure an API key.**

Choose OpenAI, Anthropic or Google. Login is API-key setup, not subscription OAuth.

4 **Check, then run a workflow.**

`--check` runs no code and calls no model.

Terminal / from the repository root — Getting started

```
# Install the CLI
npm install -g spl-lang
# No credentials needed
spl lang/examples/typed-data.md
# Or: anthropic / google
spl login openai

spl lang/examples/hi.spl --check
spl lang/examples/hi.spl --model gpt-4.1
```

In VS Code: install the SPL extension for @ completion, type-aware outputs and notebook execution.

Use a model your provider account can access. `--model` overrides workflow settings.

Tell the model *what a card is.*

Types are JavaScript values. They validate data and describe the expected shape as JSON Schema.

{js} / define the shared inputs — 01 · Contract

```
const topics = [
  'recursion', 'closures',
  'regular expressions',
];
const Topic = enumOf(...topics);
const Card = struct('Card', {
  term: Topic,
  definition: str.describe('One line'),
  description: str.describe('Several sentences'),
});
```

REQUIRED SHAPE

Card

term	one of three topics
definition	string
description	string

The model sees the fields and descriptions.
The runtime checks the actual value.

`enumOf` `struct` `list` `union` `nullable` `typedef`

Field descriptions guide the model; they do not enforce sentence counts.

Describe the task.
Save a real value.

Pick the item of `@topics` that a beginner would find hardest. Create a flashcard for it: set term to the topic, add a one-line definition and several sentences of description. `@validate[Card]` save the flashcard to `@hardest`

```
→ load_variable("Card")
→ load_variable("topics")
→ save_variable({ name: "hardest",
    value: { term, definition, description },
    schema: "Card" })
```

Illustrative output / Card

Recursion

A function solves a problem by calling itself on a smaller version of that problem.

Start with a base case that stops the calls. Then reduce the problem on each step. Beginners often find the call stack harder to picture than a simple loop.

`hardest.term` · `hardest.definition` · `hardest.description`

Use @validate[Card], not @validate[Topic], when the output must be a card object.

Good instructions help.
Checks make rules real.

PROMPT GUIDANCE / BEST EFFORT

Tell the model what you want.

`enforce` / `forbid`
Request or discourage a behavior.

Descriptions & annotations
Explain meaning, tone and intent.

`len` / "one sentence"
Guide length; not an exact validator.

RUNTIME CHECKS / ENFORCED

Verify what actually happened.

`schema` / `@validate[Card]`
Check types and required fields.

`assert(condition, message)`
Check exact business rules in code.

`restrict`
Reject library actions outside the scope.

A different way to generate data: `await ask(prompt, { schema: Card, repairs: 1 })` allows one repair attempt. A rejected blockquote tool call stops immediately.

Keep the boring parts *beautifully exact.*

{js} / use the saved Card — 03 · Output

```
assert(topics.includes(hardest.term),
  'Card must use a known topic');

const doc = markdown.document()
  .heading(hardest.term, 3)
  .paragraph(markdown.bold(
    hardest.definition))
  .paragraph(hardest.description);

await restrict('./out', () ⇒
  writeFile('./out/hardest-card.md',
    doc.toString()));
```

Native JavaScript

Control flow, arithmetic, formatting and imports. No model needed.

Exact library tools

`readFile`, `writeFile`, `shell`, `webFetch` and `webSearch`.

Scoped permissions

`fs · net · exec · mcp`

Nested scopes can only narrow access.

Not a sandbox. Restrictions cover library tools, agents and MCP calls—not direct `node:fs`, `fetch` or process APIs.

Relative paths resolve against --cwd. Always await asynchronous library calls.

Record the run.
Not a promise of repeatability.

01 / Check

Catch syntax errors.

`--check` validates without running the workflow.

02 / Record

Keep an execution history.

`--trace run.jsonl` writes an append-only journal.

03 / Reuse

Resume or replay.

Same path: recorded work.
New path: a fresh run.

Terminal / an edit-run loop — Append-only

```
spl flow.md --check
spl flow.md --trace run.jsonl
spl flow.md --trace run.jsonl

# Changed the prompt? Start fresh.
spl flow.md --trace next.jsonl
```

Keep a person in the loop.

`await human('Approve this draft?')` asks and continues. Supply answers with repeated `--human` flags; recorded answers replay without prompting.

Replay covers library effects. Your JavaScript runs again. It does not recreate deleted output files.

Keep traces private: prompts, responses, tool results and human answers can be sensitive.

Start in a document.
Grow into your stack.

01 / Editor

VS Code

Cross-block completions, `@validate[` type hints and notebook runs.

02 / Reuse

Code generation

Generate shared functions with `@CODEGEN` or `codegen()`. Traced runs cache them.

03 / Connect

MCP tools

Connect trusted servers with `--mcp mcp.json`. Allowlist agent tools.

04 / Embed

Node.js library

Import from `spl-lang`, or embed documents with `run()`.

Ordinary Node.js / same library — No workflow wrapper

```
import { ask, struct, str, enumOf }
  from 'spl-lang';

const Triage = struct('Triage', {
  label: enumOf('bug', 'feature'),
  reason: str,
});

const triage = await ask(
  'Classify: the app crashes on start',
  { schema: Triage, repairs: 1 },
);
```

A small workflow. A clear contract. *A real result.*

Let the model propose.
Let the runtime check.
Let code do the exact work.

Getting started	Setup & first run ↗
The flashcard workflow	Try the example ↗
Full documentation	Explore the guides ↗

01 Run `typed-data.md`.
Learn the contract without a model.

02 Try the flashcard agent.
See a blockquote save a checked object.

03 Change one prompt or type.
Check the file, then record a fresh trace.

04 Add one exact assertion.
Make a business rule explicit.